The Vehicle Plant in Port-Hamiltonian Form: Theory, Implementation and Validation for GRL-SNAM

Joe Rivera
Computational Visualization Center
The University of Texas at Austin

October 2026

libcvc 3.7.1 (`transfix/libcvc`) · GRL-SNAM `main` at `0e21085` (`CVC-Lab/GRL-SNAM`)

Abstract

GRL-SNAM navigates by learning the coefficients of a potential field and driving a vehicle with a bicycle controller. The vehicle under that controller is now a dynamic single-track model with Pacejka tires, load transfer and actuator lag, implemented in libcvc and twinned in GRL-SNAM's torch code. This paper writes that model as a port-Hamiltonian system on $\mathfrak{se}(2)^*$ *exactly*, at every tire slip and not only in the linear-tire regime. The dissipation is the tires' secant stiffness, which is positive semi-definite even when a tire slides. The port is the pair of axle forces, with the wheels' ground speeds as its conjugate output. Steering is not a port: it only turns the directions along which the tires push, so it does no work. Each tire's slip is measured in its wheel's frame, so the secant form leaves no remainder, and gravity on a grade is booked as a power line of its own. The integrator's step keeps per-step energy books whose residual is second order in the step. We also describe the learned stochastic port-Hamiltonian plant on $SE(2)$ (`cvc::ph`) and its energy balance, which includes the noise's power. We also describe an opt-in form of that plant in which steering is a modulation, so that only the drive supplies power, as in the analytic plant. We give the navigator's per-step energy identity, with its context and projection work at each tick. Every claim is tied to a test in libcvc or GRL-SNAM, with the tolerance the test enforces and the value measured on the released code (Section 9), so it can be checked independently.

Contents

1 Introduction **2**

2 Preliminaries **4**
2.1 Port-Hamiltonian systems 4
2.2 Planar rigid bodies and the Lie–Poisson structure 4
2.3 Where the plant sits in GRL-SNAM 4

3 The dynamic single-track vehicle **5**

4 The port-Hamiltonian form **6**
4.1 The tires' secant stiffness 6
4.2 The structure 6
4.3 Steering is not a power port 7
4.4 The speed floor 7

4.5 Grade . . . 8
4.6 Tangent versus secant . . . 8
4.7 The kinematic limit and the Jacobi identity . . . 8
4.8 Externally defined ports . . . 9

5 The discrete step and its energy books **9**
5.1 The step . . . 9
5.2 The books . . . 10
5.3 The live plant . . . 11

6 The learned plant: `cvc::ph` **11**
6.1 Model . . . 11
6.2 Gaussian processes on random Fourier features . . . 12
6.3 Energy balance, output and noise power . . . 12
6.4 The step . . . 12
6.5 Training . . . 13
6.6 Steering: a port in the reference form, a modulation by option . . . 13

7 The navigator's energy identity and context work **14**
7.1 The per-step identity . . . 14
7.2 Two checks . . . 15
7.3 Context work and projection work . . . 15
7.4 Two sets of books, not one . . . 16

8 GRL-SNAM's side **16**
8.1 The twins and their parity tiers . . . 16
8.2 What is not twinned yet . . . 16
8.3 Using the plant and its books from GRL-SNAM . . . 16

9 Validation **19**

10 Limitations and future work **19**

A Reproducing the validation tables **24**
A.1 What was run . . . 24
A.2 libcvc . . . 24
A.3 What the libcvc tests print . . . 25
A.4 GRL-SNAM . . . 26

B Equation-to-code map **27**

1 Introduction

GRL-SNAM [1] treats navigation as motion in a learned energy landscape. A small network predicts the coefficients $(\alpha_j, \beta, \gamma)$ of a potential: barrier weights per obstacle, a goal spring and a damping. A controller turns the potential's force into steering and acceleration commands for a vehicle. GRL-SNAM's own guide to the C++ port (`docs/cvc-nav-and-grl-snam-guide.md`, "A note on the vocabulary") is careful about what this does and does not buy. The deployed navigator is a learned-coefficient potential field with viscous damping, on top of a kinematic bicycle and a

pure-pursuit controller. No Hamiltonian is carried through it, and no energy or passivity argument transfers to it.

This paper is about the layer under the navigator: the vehicle. Since libcvc 3.7.0, a dynamic single-track vehicle (the plant) can replace the kinematic update where a policy asks for it. libcvc 3.7.1 writes that vehicle as a port-Hamiltonian (pH) system with a genuine Hamiltonian H, momenta, a positive semi-definite dissipation, and a power port. It keeps books of where the vehicle's kinetic energy goes, step by step. The same release adds two related pieces: a learned stochastic pH model of a vehicle, and per-step energy books for the navigator's point-mass surrogate. Across the navigator's ticks those books include the potential's changes of context.

Contributions.

1. An exact pH form of the dynamic single-track vehicle, valid at every slip (Proposition 1). The dissipation is built from the tires' secants, the port is the axle forces, and steering is shown not to be a power port (Section 4).

2. The discrete step's energy books, with a residual that is second order per step (Section 5), and a live plant that keeps them for every vehicle in a navigation world.

3. The learned plant `cvc::ph`: its structure, its energy balance including the noise power, its integrator and its training objective (Section 6). It also has an opt-in form in which steering modulates the model instead of supplying power, as it does in the analytic plant (Section 6.6).

4. The navigator's per-step energy identity, the two checks built on it, and the context work at tick boundaries (Section 7).

5. GRL-SNAM's side: the torch twins and their parity tiers, what is not twinned yet, and how a GRL-SNAM rollout or trainer can use the plant and its books (Section 8).

6. A validation table that maps each claim to a test, its tolerance, the measured value and the commit that added it (Section 9).

What is not claimed. The closed loop of navigator, controller and plant is not port-Hamiltonian. The controller is a path follower, not an energy-shaping controller (Section 10). Neither the analytic plant's integrator nor the learned plant's is structure-preserving. Both keep books with a residual instead. The learned plant's filter-based evidence bound is not yet implemented, and the four-wheel vehicle's pH form is not yet written.

Versions. Every statement about code refers to libcvc 3.7.1 or later and GRL-SNAM `main` at `0e21085`, unless it names a later pull request.

- The dynamic plant itself is transfix/libcvc #714, released in 3.7.0.
- The learned plant is #758, released in 3.7.1.
- The pH form and the books are #831, #832 and #833, also released in 3.7.1.
- Steering as a modulation of the learned plant (Section 6.6) is #851, which is later than 3.7.1.
- The front slip in its wheel's frame (Sections 3 and 4.4) is #852, also later than 3.7.1. It changes the plant's outputs; its golden for the pH form is #848's.

The measurements in Section 9 were taken on libcvc `master` at `c1f104c0`, whose `nav`, `ph` and `physics` sources and tests are identical to v3.7.1's, except for the rows from #851 and #852, which were taken on their branches.

2 Preliminaries

2.1 Port-Hamiltonian systems

An input-state-output pH system [7] with state x, stored energy $H(x)$, input u and output y is

$$\dot{x} = \big(\mathsf{J}(x) - \mathsf{R}(x)\big)\nabla H(x) + \mathsf{G}(x)\,u, \qquad y = \mathsf{G}(x)^\top \nabla H(x), \tag{1}$$

with $\mathsf{J} = -\mathsf{J}^\top$ (the interconnection) and $\mathsf{R} = \mathsf{R}^\top \succeq 0$ (the dissipation). Along solutions, $\dot{H} = -\nabla H^\top \mathsf{R} \nabla H + y^\top u \le y^\top u$, so the system is passive from the port (u, y), with H as storage. The structure tells you where energy goes: J moves it without loss, R dissipates it, and the port $y^\top u$ is the only way it enters from outside. If R may depend on the input, nothing in this changes, provided it is positive semi-definite for every input value.

2.2 Planar rigid bodies and the Lie–Poisson structure

A planar rigid body has pose $q = (x, y, \psi) \in SE(2)$ and body twist $v = (v_x, v_y, r) \in \mathfrak{se}(2)$: the velocity of its centre of gravity in the body frame, and its yaw rate. With mass matrix $M = \mathrm{diag}(m, m, I_z)$ the momenta are $p = Mv \in \mathfrak{se}(2)^*$, and the kinetic energy is $H(p) = \frac{1}{2} p^\top M^{-1} p$, so $\nabla_p H = v$. The body-frame equations of motion of a free body are the Euler–Poincaré equations $\dot{p} = \mathrm{ad}_v^*\, p + f$, with f the applied body wrench and

$$\mathrm{ad}_v^*\, p = \mathsf{J}(p)\, v, \qquad \mathsf{J}(p) = \begin{pmatrix} 0 & 0 & p_y \\ 0 & 0 & -p_x \\ -p_y & p_x & 0 \end{pmatrix}. \tag{2}$$

$\mathsf{J}(p)$ is skew-symmetric and linear in p: it is the Lie–Poisson structure of $\mathfrak{se}(2)^*$ [6], so it satisfies the Jacobi identity. With M diagonal and equal masses on v_x and v_y, $\mathsf{J}(p)v = (mv_y r,\, -mv_x r,\, 0)$. These are the familiar $+rv_y$ and $-rv_x$ terms of a vehicle's body-frame equations.

2.3 Where the plant sits in GRL-SNAM

GRL-SNAM's navigator evaluates a potential U_{nav} at the agent's position o,

$$U_{\mathrm{nav}}(o) = \tfrac{\beta}{2}\|o - g\|^2 + \textstyle\sum_j \alpha_j\, b(d_j(o)) + (\text{material and terrain terms}), \tag{3}$$

with b the IPC barrier of each obstacle's clearance d_j, and g the current goal or carrot. Damping $-\gamma v$ is dissipation, not part of U. An external force, where one is used, is a port, not part of U either. libcvc evaluates U_{nav} along every rollout (`inc/cvc/nav/detail/nav_potential.h`), and GRL-SNAM's torch twin (`grl_snam/nav_potential.py`, CVC-Lab/GRL-SNAM#133) is held to it. The controller (`bike_control` in `diff_rollout.h`; `sdf_nav.bike_control` in GRL-SNAM) turns the force $-\nabla U_{\mathrm{nav}}$, plus any external force, into a steering command δ_{cmd} and an acceleration command a_{cmd}. A plant then moves the vehicle. The default plant is the kinematic bicycle, $\dot{\psi} = (v/L)\tan\delta$. The dynamic plant of this paper takes the same commands, so a policy can train through tires, load transfer and actuator lag with the controller and its adjoint unchanged.

3 The dynamic single-track vehicle

The plant is `cvc::nav::diff::dyn_plant` (`inc/cvc/nav/detail/dyn_bike.h`). It is written once over a scalar type: float in training, double in the tests, and a forward-mode dual number for its adjoint. Its torch twin is `grl_snam.dyn_bike.dyn_plant`. The model is the standard single-track ("bicycle") vehicle [4] with Pacejka lateral forces [3]. We summarize what the pH form needs.

State and parameters. The state is $(x, y, \psi, v_x, v_y, r, \delta, a)$: the rear axle's position, the heading, the body twist at the centre of gravity, the road-wheel steering angle, and the drive or brake force per unit mass. The last two sit behind first-order actuators. The parameters are the mass m, the yaw inertia I_z, the distances l_f, l_r from the centre of gravity to the axles ($L = l_f + l_r$), the centre of gravity's height h_{cg}, the ground's friction μ, Pacejka factors B_f, B_r and C, drag c_d, rolling resistance c_{rr}, the drive and brake split between the axles, and a speed floor $\varepsilon > 0$.

Loads and grip. Quasi-static longitudinal load transfer gives axle loads $F_{z,f} = m(gl_r - ah_{\mathrm{cg}})/L$ and $F_{z,r} = m(gl_f + ah_{\mathrm{cg}})/L$, each floored at a small fraction of mg. The drive or brake force ma splits between the axles, and each share is saturated smoothly at $\mu F_{z,w}$. These are the axle forces $F_{x,f}, F_{x,r}$. What grip they leave for cornering, P_w, follows from the friction ellipse with the longitudinal force first. Both depend on the actuator state a.

Directions. Each tire's lateral force reaches the twist's coordinates (v_x, v_y, r) along n_w, and each axle's drive or brake force along t_w:

$$n_f(\delta) = \begin{pmatrix} -\sin\delta \\ \cos\delta \\ l_f\cos\delta \end{pmatrix}, \quad n_r = \begin{pmatrix} 0 \\ 1 \\ -l_r \end{pmatrix}, \quad t_f(\delta) = \begin{pmatrix} \cos\delta \\ \sin\delta \\ l_f\sin\delta \end{pmatrix}, \quad t_r = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}. \tag{4}$$

So $n_w^\top v$ is the sideways velocity of that axle's contact patch in its wheel's frame, and $t_w^\top v$ is its ground speed along the wheel.

Slips. Each slip is measured in its wheel's frame, as in the four-wheel model. With $w_w = n_w^\top v$ the axle's sideways velocity there, $w_f = \cos\delta\,(v_y + l_f r) - \sin\delta\, v_x$ and $w_r = v_y - l_r r$, and $u_w = \sqrt{(t_w^\top v)^2 + \varepsilon^2}$ its floored forward speed, the slip angles are

$$\alpha_w = -\operatorname{atan2}(w_w, u_w), \qquad w \in \{f, r\}. \tag{5}$$

The floor keeps them smooth through zero speed. At the rear, $u_r = \bar{u} = \sqrt{v_x^2 + \varepsilon^2}$, the floored speed that also smooths drag and rolling resistance. The lateral forces are Pacejka's, $F_{y,w} = P_w \sin(C \arctan(B_w \alpha_w))$, with the shape factor $C \in (1, 2)$, as the parameter's documentation requires. Before libcvc #852 the front slip was $\varsigma\delta - \operatorname{atan2}(v_y + l_f r, \bar{u})$, with $\varsigma = v_x/\bar{u}$, which left the remainder of Section 4.4.

Equations of motion. With θ the sine of the slope along the heading (positive uphill), and $c_x(\bar{u}) = c_d\bar{u} + c_{rr}mg/\bar{u}$ the drag and rolling resistance acting on v_x, the body-frame Newton–Euler equations are

$$\begin{aligned} m(\dot{v}_x - r v_y) &= F_{x,f}\cos\delta - F_{y,f}\sin\delta + F_{x,r} - c_x v_x - mg\theta, \\ m(\dot{v}_y + r v_x) &= F_{x,f}\sin\delta + F_{y,f}\cos\delta + F_{y,r}, \\ I_z\dot{r} &= l_f\big(F_{x,f}\sin\delta + F_{y,f}\cos\delta\big) - l_r F_{y,r}. \end{aligned} \tag{6}$$

In momenta and the directions above, (6) reads

$$\dot{p} = \mathsf{J}(p)v + t_f F_{x,f} + t_r F_{x,r} + n_f F_{y,f} + n_r F_{y,r} - c_x v_x\, e_x - mg\theta\, e_x. \tag{7}$$

Left out on purpose: wheel spin, suspension, lateral load transfer and contact loss. The four-wheel model (Section 10) adds some of these.

4 The port-Hamiltonian form

4.1 The tires' secant stiffness

The key to an exact form is to write each lateral force as a stiffness times a velocity, in a way that stays valid past the tire's peak. A tangent stiffness does not. The secant through the force's root does.

Lemma 1 (Secant stiffness)**.** *With the slips* (5)*, $B_w > 0$, $C \in (0, 2)$ and $P_w \geq 0$, the secant stiffness*

$$S_w = -\frac{F_{y,w}}{w_w} \qquad (w_w \neq 0), \tag{8}$$

continued at the root by the slope there, $S_w = P_w C B_w u_w/(u_w^2 + w_w^2)$, is non-negative at every slip, every speed and every steering angle, sliding included. So $F_{y,w} = -S_w\, w_w = -S_w\, n_w^\top v$.

Proof. Since $u_w \geq \varepsilon > 0$, $\alpha_w = -\operatorname{atan2}(w_w, u_w)$ has the sign of $-w_w$ and vanishes exactly at $w_w = 0$: each slip's root is where its contact patch has no sideways velocity in its wheel's frame. Since $C \arctan(B_w \alpha_w) \in (-C\pi/2,\, C\pi/2) \subset (-\pi, \pi)$, the force $P_w \sin(C \arctan(B_w \alpha_w))$ has the sign of α_w, or is zero. Hence $F_{y,w}$ and w_w have opposite signs, and $S_w \geq 0$. At the root, $-\partial F_{y,w}/\partial w_w$ (at fixed u_w) is $P_w C B_w u_w/(u_w^2 + w_w^2) \geq 0$. □

The implementation is `dyn_secant_w`. It takes the root branch when $(B_w \alpha_w)^2 < 10^{-8}$, so there $F_{y,w} = -S_w w_w$ holds to second order in the slip, not exactly. That rounding is the only remainder; it is booked on its own line (Section 4.4). A Pacejka force with $C < 2$ never changes sign against its slip. A tire past its peak gives less force for more slip, but never pushes the slip further.

4.2 The structure

Define

$$\mathsf{R}(v; \delta, a) = c_x(\bar{u})\, e_x e_x^\top + S_f\, n_f n_f^\top + S_r\, n_r n_r^\top, \tag{9}$$

$$\mathsf{G}(\delta) = \begin{bmatrix} t_f & t_r \end{bmatrix}, \qquad u = (F_{x,f},\, F_{x,r}), \qquad y = \mathsf{G}(\delta)^\top v, \tag{10}$$

$$f_\varepsilon = n_f(F_{y,f} + S_f\, n_f^\top v) + n_r(F_{y,r} + S_r\, n_r^\top v). \tag{11}$$

R depends on the actuator state a through the grips P_w in S_w. f_ε is what the secant dissipation leaves of the tires' actual forces. By Lemma 1 it is zero off the secants' root branch, and that branch's rounding on it (Section 4.4). In code these are `dyn_ph_J`, `dyn_ph_R`, `dyn_ph_G`, `dyn_ph_input` and `dyn_ph_floor`. Their sum is `dyn_ph_rhs`, and `dyn_ph_rates_at` evaluates the power balance at a state.

Proposition 1 (The dynamic single-track vehicle is port-Hamiltonian at every slip)**.** *Under the conditions of Lemma 1, with $c_d, c_{rr} \geq 0$, the vehicle's equations of motion* (6) *are exactly*

$$\dot{p} = \big(\mathsf{J}(p) - \mathsf{R}(v; \delta, a)\big)v + \mathsf{G}(\delta)\, u + f_\varepsilon - mg\theta\, e_x, \qquad y = \mathsf{G}(\delta)^\top v, \tag{12}$$

with $H = \frac{1}{2}p^\top M^{-1}p$, $\mathsf{J} = -\mathsf{J}^\top$ *the Lie–Poisson structure of* $\mathfrak{se}(2)^*$, *and* $\mathsf{R} = \mathsf{R}^\top \succeq 0$ *at every state, slip and input. Consequently*

$$\dot{H} = -v^\top \mathsf{R} v + y^\top u + v^\top f_\varepsilon - mg\theta\, v_x. \tag{13}$$

The vehicle is passive with respect to the supply $y^\top u + v^\top f_\varepsilon - mg\theta v_x$. *Since* f_ε *is zero but for the secants' rounding at the root, on level ground it is passive from the drive port* (u, y) *alone, at every speed and every floor* ε.

Proof. By (11), $n_f F_{y,f} + n_r F_{y,r} = -(S_f n_f n_f^\top + S_r n_r n_r^\top)v + f_\varepsilon$. Substituting this into (7), with $t_f F_{x,f} + t_r F_{x,r} = \mathsf{G}u$ and $c_x v_x\, e_x = c_x\, e_x e_x^\top v$, gives (12). $\mathsf{J}(p)$ is skew and is the Lie–Poisson tensor (2), so the Jacobi identity holds. R is a combination of outer products with coefficients $c_x \geq 0$, $S_f \geq 0$ and $S_r \geq 0$ (Lemma 1), so it is symmetric positive semi-definite. Finally $\dot{H} = v^\top \dot{p}$, $v^\top \mathsf{J} v = 0$ and $v^\top \mathsf{G} u = y^\top u$. □

4.3 Steering is not a power port

In (12) the steering angle appears only inside n_f and t_f, and through them in the front slip and its secant. It turns the directions along which the front tire pushes, so it modulates the structure. It is not an effort with a conjugate flow, and it does no work on the body. The power that drives the body is $y^\top u = F_{x,f}\, t_f^\top v + F_{x,r}\, t_r^\top v$: each axle's force times its wheel's ground speed. With no axle force, nothing is supplied, however the wheel turns.

This matters because the natural linear-tire pH form gets it wrong, and libcvc's own first pH form (transfix/libcvc#748, replaced in #831) did. With linear tires $F_{y,f} \approx C_f \alpha_f$ and small angles, it is tempting to take $u = (F_x, \delta)$, with input column $C_f(0, 1, l_f)^\top$ for δ. The conjugate output of δ is then $C_f(v_y + l_f r)$, and the supplied power contains $C_f \delta (v_y + l_f r)$. This has no sign, so that model can gain kinetic energy from steering alone, with no drive force. It is the first-order truncation in δ of (9). Expanding $n_f \approx (-\delta, 1, l_f)$ and, at small slip, $S_f \approx C_f / u_f \approx C_f / \bar{u}$, the v_x-column of $\frac{C_f}{\bar{u}} n_f n_f^\top$ contributes $+C_f \delta\, (v_x/\bar{u})\, (0, 1, l_f)^\top$ to $-\mathsf{R}v$, which is $C_f \delta\, (0, 1, l_f)^\top$ when $v_x \approx \bar{u}$. The truncation moves that term out of R and calls it an input, and it drops the cornering drag $-F_{y,f} \sin\delta$ on v_x. In the modulated form, the front tire's power $F_{y,f}\, n_f^\top v$ is the dissipation $-S_f (n_f^\top v)^2 \leq 0$, exactly but for the root branch's rounding. Steering only redirects energy.

4.4 The speed floor

The floor ε enters the slips (5) only through their forward speeds u_w, which keep them smooth through zero speed. It does not move their roots: each slip vanishes where $n_w^\top v = 0$, which is where the secant form puts it. So off the secants' root branch $f_\varepsilon = 0$ exactly, and on it f_ε is that branch's rounding. The floor line of the books is kept, and measures it. The test `DynBikePH.TheFloorTermIsRoundOff` finds its worst share of the flow at most 2.2×10^{-16} at every floor from $\varepsilon = 0.5$ down to 10^{-9} m/s.

This holds since libcvc #852, which moved the front slip into its wheel's frame. Before it, the front slip $\varsigma\delta - \operatorname{atan2}(v_y + l_f r, \bar{u})$, with $\varsigma = v_x/\bar{u}$, had its root at $v_y + l_f r = \bar{u}\tan(\varsigma\delta)$, not where $n_f^\top v = 0$ ($v_y + l_f r = v_x \tan\delta$). The remainder was $f_\varepsilon = S_f'(\bar{u}\tan(\varsigma\delta) - v_x \tan\delta)\, n_f$, with S_f' that slip's secant in $v_y + l_f r$, and R's front term was $(S_f'/\cos\delta)\, n_f n_f^\top$, positive semi-definite only for $|\delta| < \pi/2$. The bracket is $O(\varepsilon^2/|v_x|)$, and the floor's worst share of the flow fell as ε^2: 5.4×10^{-4}, 5.4×10^{-6} and 5.4×10^{-8} at $\varepsilon = 0.5$, 0.05, 0.005 m/s. Outside a crawl the change moves the plant little: the vector field, the books' order and the kinematic limit of Table 2 measure the same before and after it.

4.5 Grade

On a slope, gravity along the heading does work $-mg\theta\, v_x$. The plant has no side slope and no height state, so H carries no potential energy. The grade enters as an external power line, booked separately. (A plant with a height state would move it into H as mgz. The four-wheel plant's suspension, Section 10, is the natural place.)

4.6 Tangent versus secant

Past a tire's peak, $|B_w\alpha_w| > \tan(\pi/(2C))$, its force falls as its slip grows. The *tangent* dissipation, minus the derivative of the tires' lateral force Q_y and yaw moment Q_r in (v_y, r), is

$$D_t = -\frac{\partial(Q_y, Q_r)}{\partial(v_y, r)} = k_f\, aa^\top + k_r\, bb^\top, \qquad a = (1, l_f),\ b = (1, -l_r), \tag{14}$$

with k_w each tire's $\partial F_{y,w}/\partial\alpha_w$ times its slip's sensitivity to (v_y, r) along a or b. At the rear that is $u_r/(u_r^2 + w_r^2)$. At the front, the sideways velocity w_f and the forward speed $t_f^\top v$ (floored to u_f) both move along a, as $\cos\delta$ and $\sin\delta$, and the force reaches the body through $\cos\delta$, so

$$k_f = \frac{\partial F_{y,f}}{\partial\alpha_f}\,\frac{\cos\delta\,(\cos\delta\,u_f - \sin\delta\,w_f\,t_f^\top v/u_f)}{u_f^2 + w_f^2}. \tag{15}$$

In code this is `dyn_ph_tangent_D`; `DynBikePH.TheTangentIsTheTiresDerivative` checks it against central differences of the tires' lateral force and moment, to 1.4×10^{-7} of its scale. Since a and b are independent, $D_t \succeq 0$ exactly when both $k_w \geq 0$: when both tires are below their peak (and the front's slip angle is under about $90° - |\delta|$, where its sensitivity changes sign). Past the peak, the linearized motion can be locally unstable, which is sliding. That is a statement about stability, not about energy. The secant (9), the dissipation in the energy balance (13), stays positive semi-definite. So a positive semi-definite dissipation of the form $-D(v)v$ can represent sliding: its gain falls with slip. What it cannot represent is a tire that supplies energy, and a Pacejka tire with $C < 2$ never does.

On the tests' full-size utility-vehicle preset (`DynBikePH.TheSecantStaysPSDWhereTheTangentDoesNot`), with steer ramps to the lock and swerves at 8–20 m/s:

- The vehicle understeers: the front tire passes its peak first.
- The tangent is positive semi-definite in every sample below about half the grip, $0.5\mu g$. Above it, it is indefinite in up to half the samples of a lateral-g bin.
- The rear tire never passes its peak.
- The secant is positive semi-definite in every sample, at $\mu = 0.9$ and at $\mu = 0.6$.

This bears on any learned model that constrains $D(v) \succ 0$, such as the one in Section 6. The constraint does not rule out sliding.

4.7 The kinematic limit and the Jacobi identity

The kinematic bicycle imposes a no-slip constraint at each axle. That constraint is not integrable, so the reduced bracket is only almost-Poisson and the Jacobi identity fails [8]. The dynamic vehicle replaces the constraint by tire forces, its J is a genuine Lie–Poisson structure, and the kinematic bicycle is its stiff limit: $S_w \to \infty$ along n_f and n_r drives $n_w^\top v$ to zero. The test `DynBike.Kinematic`

`LimitFollowsBikeStep` checks the limit numerically. It uses stiff tires and puts the mass at the rear axle, as the kinematic bicycle has it. Under the kinematic bicycle's commands, the dynamic vehicle then stays within 0.011 (normalized units) of the kinematic path over 200 ticks, with a mean path length of 5.6.

4.8 Externally defined ports

The form (12) accepts further ports. Suppose an external generalized force acts on the body: a tow, wind, a contact, or a force defined by some other model. It enters through its own input map, $\mathsf{G}_e u_e$, and has its own conjugate output $y_e = \mathsf{G}_e^\top v$. The balance gains one line, $y_e^\top u_e$, which is booked as supplied power:

$$\dot{p} = (\mathsf{J} - \mathsf{R})v + \mathsf{G}u + \mathsf{G}_e u_e + f_\varepsilon - mg\theta e_x, \qquad \dot{H} = -v^\top \mathsf{R} v + y^\top u + y_e^\top u_e + v^\top f_\varepsilon - mg\theta v_x. \quad (16)$$

Nothing else changes: J and R are untouched, and passivity holds with the added supply. For a force $F = (F_X, F_Y)$ given in the world frame and applied at the centre of gravity, the body wrench is $(R(\psi)^\top F, 0)$. So

$$\mathsf{G}_e = \begin{pmatrix} R(\psi)^\top \\ 0 \quad 0 \end{pmatrix}, \qquad y_e = R(\psi) \begin{pmatrix} v_x \\ v_y \end{pmatrix},$$

and y_e is the centre of gravity's world velocity. The supplied power is $F \cdot \dot{x}_{\mathrm{cg}}$, as it should be. Applied at a body point (ℓ_x, ℓ_y) instead, the yaw row of G_e becomes $(-\ell_y, \ell_x)R(\psi)^\top$, the moment arm. A port that stores energy, such as a spring, belongs in H instead.

Worked example: a navigation force. GRL-SNAM and libcvc allow an external navigation force F_{ext}, with its spatial Jacobian. It might come from another field, a tow or a learned term. It is joined to the navigator's conservative force (`ext_in` in `bike_control`; `grl_snam.ext_rollout`, `material_ext`), and it reaches the two models differently.

- On the navigator's point-mass surrogate, F_{ext} is a port force on the point mass. The per-step books (Section 7) carry it as the port work $W_{\mathrm{port}} = \frac{h}{2} F_{\mathrm{ext}} \cdot (v + v')$. Left out, it is work of order h, against a bound of order h^2. The test `NavPotential.UnloggedForceRaisesTheAlarm` sees exactly that: 229 of 240 steps raise the alarm.
- On the vehicle, F_{ext} acts through the controller. It changes $(\delta_{\mathrm{cmd}}, a_{\mathrm{cmd}})$, so it reaches the body only through the axle forces, which are already booked as $y^\top u$, and through steering, which does no work. The plant's books need no new line: the external force's energy shows up as drive or brake. A model in which the force acts on the body directly, as a physical force, would add the column G_e above, with F_{ext} as u_e. libcvc does not implement such a column today.

5 The discrete step and its energy books

5.1 The step

One step of length h (`dyn_plant`) from the commands $(\delta_{\mathrm{cmd}}, a_{\mathrm{cmd}})$ and the grade has these parts:

1. *Actuators.* The steering follows its command with an exact first-order lag, $1 - e^{-h/\tau}$, with its change limited smoothly to a slew rate times h. The force follows its command with its own lag. Their outputs (δ_1, a_1) are the inputs the rest of the step applies.

2. *Loads and axle forces* from a_1, as in Section 3.

3. *Speed, predicted* from the axle forces, drag, rolling resistance and grade alone. Lateral forces are not read before step 4: in the stiff limit they are constraint forces.

4. *Lateral and yaw, implicit.* (v_y, r) advance by implicit Euler at the predicted speed. The step is solved by a few Newton-like iterations, each linearizing the tires by their secants at its iterate. A sliding tire then pulls its slip toward zero, never past it, and stiff tires stay stable at the navigator's 0.06 s tick.

5. *Speed, corrected* with the front lateral force the implicit step applied, then *clamped* to the vehicle's speed range.

6. *Low-speed blend.* Below a blend speed, a sigmoid in $|v_x|$ blends (v_y, r) toward the kinematic bicycle's, where slip angles mean nothing.

Then the pose advances. It does not enter H. The step is first order in h.

5.2 The books

`dyn_plant_books` runs exactly that step and returns, in the struct `dyn_ph_books`,

$$\Delta H = \underbrace{\widehat{\int} y^\top u}_{\text{supplied}} - \underbrace{\widehat{\int} v^\top \mathsf{R} v}_{\text{dissipated}} + \underbrace{\widehat{\int} v^\top f_\varepsilon}_{\text{floor}} \underbrace{- \widehat{\int} mg\theta v_x}_{\text{grade}} + W_{\text{lim}} + r_h. \tag{17}$$

Each $\widehat{\int}$ is the trapezoid, over h, of that power. Its two ends are the step's start (the start twist, with the inputs (δ_1, a_1) the step applied) and the twist the step reached *before* its limiters, the speed clamp and the low-speed blend. W_{lim} is the kinetic energy those limiters then changed, booked exactly as the difference of H across them. r_h is what is left, the integrator's residual. The step's arithmetic does not depend on the books: `dyn_plant` records the pre-limiter twist only when asked, and the books left `DynBike.Golden`'s outputs unchanged.

Proposition 2 (The books close to second order)**.** *Where the vector field is smooth along the step, $r_h = O(h^2)$ per step, so over a fixed horizon T the summed residual is $O(h)$.*

Argument. Before its limiters, the step is a consistent first-order method: implicit Euler on (v_y, r), and predicted-then-corrected Euler on v_x with the same forces. So the pre-limiter twist differs from the exact flow's twist at $t + h$ by $O(h^2)$. Along the exact flow, $H(t+h) - H(t) = \int_t^{t+h} \dot{H}$ by (13), and the trapezoid of a smooth integrand has error $O(h^3)$. Replacing the exact end state by the step's changes H by $O(h^2)$ and the trapezoid by $h \cdot O(h^2)$. The limiters' energy is booked exactly. Hence $r_h = O(h^2)$. The finite number of Newton iterations, and the branch switches (load floor, saturation, the secant's root branch), are part of r_h, not exceptions to it. □

`DynBikePH.StepBooksCloseToSecondOrder` checks this on a swerve past the tire's peak, with the throttle working and the actuators lagging, at $\mu = 0.6$. As h halves from 0.02 to 0.0025 s, the mean per-step $|r_h|$ falls by 3.83, 3.92 and 3.95. Over 6 s at $h = 0.01$ s the summed residual is 1.2×10^{-3} of the energy that moved (the flow: the sum of the magnitudes of every line). Braking hard to a stop in a turn makes the clamp and the blend act. The books charge them (for example -358 J at $h = 0.01$ s), and the rest closes to 4×10^{-6} of the flow.

5.3 The live plant

`cvc::physics::dyn_bike_plant` (libcvc #833) makes the same vehicle a plant that a navigation world (`cvc::nav::sim_world`) drives. It sits beside the kinematic bicycle and the rigid-body engine's vehicles, and needs no physics world. Agent i drives one of a list of vehicle descriptions, with parameters from `nav_plant::dyn_bike_si_of`, so the dynamic and rigid-body plants read one description. Each tick of length Δt runs $\lceil \Delta t/h \rceil$ steps under the held command, with $h = 0.01$ s by default (six steps at the navigator's 0.06 s tick), in SI units. It books every step through `dyn_plant_books`. The books are kept per tick (`last_tick`) and since the agent was attached (`books`):

$$K - K_0 = \text{drive} - \text{brake} - \text{dissipated} + \text{floor} + \text{grade} + \text{limiter} + \text{residual}, \tag{18}$$

where drive and brake are the positive and negative parts of each step's supplied term, split step by step. The plant is flat (grade zero) and has no world to collide with. It is selected through `plant_host` as `"dynamic"`, or `"dynamic:<presets>"` to cycle vehicles. It is the plant that the coefficient trainer's `rollout_kind::dynamic` trains through, so a policy trained there is evaluated on its own model.

6 The learned plant: `cvc::ph`

`cvc::ph::se2_vehicle` (`inc/cvc/ph/se2_vehicle.h`) is a native C++ port of an $SE(2)$ vehicle model built on the stochastic port-Hamiltonian Gaussian processes of McLennan, Rapolu and Bajaj [2]. Their JAX code, `CVC-Lab/LieSPHGP`, models an $SO(3)$ pendulum, $SE(3)$ quadrotors and an $SE(3)$ underwater vehicle. It has no $SE(2)$ vehicle.

The port follows a separate $SE(2)$ vehicle package built on that code at commit `65db994`. That package is not part of the public repository. libcvc's golden for the port (`ph_se2_golden.json`) was generated once from it, and its provenance records "`CVC-Lab/LieSPHGP @ 65db994 + the SE(2) package`". The package follows LieSPHGP's underwater-vehicle model reduced to $SE(2)$, with two differences: its input map reads the twist, where LieSPHGP's reads the pose, and its inputs are a vehicle's drive and steering.

The model contains no physical constant: everything is learned from logged runs of a vehicle. In libcvc those runs come from `cvc::physics::plant_log`. It records the vehicles of a navigation world driving the rigid-body engine's plant (`cvc::physics::nav_plant`, on Jolt Physics).

6.1 Model

The state is (x, y, ψ, v) with $v = (v_x, v_y, r)$. The inputs u are the logged commands divided by their training RMS. With $p = Mv$ and $H = \frac{1}{2} p^\top M^{-1} p + V(x, y)$,

$$\begin{aligned} d(x, y) = R(\psi)(v_x, v_y)\, dt, \qquad d\psi = r\, dt, \qquad dv = M^{-1} dp + \Sigma\, dW, \\ dp = \Big(\mathrm{ad}_v^*\, p - \begin{pmatrix} R(\psi)^\top \nabla V \\ 0 \end{pmatrix} - D(v)\, v + G(v)\, u \Big) dt, \end{aligned} \tag{19}$$

with $\Sigma = \mathrm{diag}(\sigma_1, \sigma_2, \sigma_3)$. Each piece is a learned level plus a Gaussian-process residual:

- $M^{-1} = \mathrm{spd}(\ell_M)$, constant: a rigid body's inertia.
- $D(v) = \mathrm{spd}\left(\ell_D + \mathrm{GP}_D(v/s_v)\right)$, positive definite at every twist: it can only dissipate.
- $G(v) = \ell_G + \mathrm{GP}_G(v/s_v)$, a $3 \times n_u$ input map.

- $V(x,y) = \ell_V \cdot (x,y) + \mathrm{GP}_V\big((x,y)/\lambda_V\big)$, the potential.

Here s_v is the RMS of each twist channel over the training data. Here $\mathrm{spd}(\ell) = LL^\top$, where L is lower triangular with diagonal $e^{\ell_1}, e^{\ell_2}, e^{\ell_3}$ and off-diagonal ℓ_4, ℓ_5, ℓ_6 (`spd_from_levels`, `mat3.h`), so the result is symmetric positive definite for every value of the six levels. Only $M^{-1}D$, $M^{-1}G$ and $M^{-1}\nabla V$ are identifiable from trajectories, plus M's shape through $\mathrm{ad}_v^* p$.

6.2 Gaussian processes on random Fourier features

Each GP is $\mathrm{GP}(z) = \phi(z)^\top W$ on n random Fourier features [5] of a Matérn-ν kernel with length scale λ (`inc/cvc/ph/rff.h`):

$$\phi_k(z) = \sqrt{2/n}\,\cos(\omega_k \cdot z + b_k), \qquad \omega_k = \frac{w_k}{\lambda\sqrt{s_k/(2\nu)}}, \quad w_k \sim \mathcal{N}(0,I),\ s_k \sim \chi^2_{2\nu},\ b_k \sim \mathcal{U}(0,2\pi). \tag{20}$$

These are the Student-t spectral samples of the Matérn kernel. Every derivative is bounded and in closed form, which the reverse pass uses. The weights have a mean-field Gaussian posterior, $q(W) = \mathcal{N}(\mathrm{mean}, \mathrm{diag}\, e^{2\log \mathrm{std}})$. The defaults are $n = 64$, $\nu = 5/2$ and $\lambda = 1$. The library's own generator (xoshiro256** with written-out normal and gamma draws) draws the same features from a seed on every compiler.

6.3 Energy balance, output and noise power

Along the drift of (19), $\mathrm{ad}_v^* p$ does no work, and the potential's force returns exactly what moving through V costs. So

$$\dot H = -v^\top D(v)\, v + y^\top u, \qquad y = G(v)^\top v. \tag{21}$$

`output()` returns y, so $y^\top u$ is the supplied power. The noise is added to the twist, so it supplies energy too, on average. Since $dH = v^\top M\, dv + \frac{1}{2} dv^\top M\, dv$ (plus the position terms) and Σ is constant, so the Itô and Stratonovich forms agree,

$$\mathbb{E}[dH] = \Big(- v^\top D v + y^\top u + \tfrac{1}{2}\,\mathrm{tr}\,(\Sigma^\top M \Sigma)\Big) dt. \tag{22}$$

`noise_power()` returns $\frac{1}{2}\,\mathrm{tr}(\Sigma^\top M \Sigma)$. Pathwise, the drift alone is passive. The diffusion's power is booked as its own line, not attributed to D.

6.4 The step

One step of size h is "Lie-IMEX": the damping is implicit and everything else is explicit. With $e = M^{-1}(\mathrm{ad}_v^* p - [R^\top \nabla V; 0] + Gu)$ and $K = M^{-1}D(v)$ evaluated at each stage, and the twist's noise increment $\eta = \sqrt{h}\,\Sigma z$,

$$\begin{aligned} &(I + hK_1)\, v_p = v + h\, e_1 + \eta, && (x,y)_p = (x,y) + hR(\psi)(v_x, v_y), && \psi_p = \psi + hr, \\ &(I + \tfrac{h}{2}K_2)\, v' = v + \tfrac{h}{2}(e_1 + e_2 - K_1 v) + \eta, && (x,y)' = (x,y) + \tfrac{h}{2}\big(R(\psi)v_{xy} + R(\psi_p)v_{p,xy}\big), && \end{aligned} \tag{23}$$

and $\psi' = \psi + \frac{h}{2}(r + r_p)$. It is a Heun predictor–corrector with implicit damping, and so second order. Since $SO(2)$ is abelian, the heading stays a continuous angle without a group-valued update. It is not structure-preserving and keeps no discrete energy identity. The step is templated on the scalar, so the same code runs in float and in double.

6.5 Training

`cvc::ph::se2_rollout_objective` (`inc/cvc/ph/se2_train.h`) is the rollout loss of the $SE(2)$ package's trainer:

$$\mathcal{L} = \text{rollout}(\text{windows} \mid W) + \beta \, \frac{\mathrm{KL}(q \,\|\, p)}{6 N_{\text{obs}}}, \qquad W = \text{mean} + e^{\log \text{std}} z, \; z \sim \mathcal{N}(0, I). \tag{24}$$

From each window's first sample, the model is rolled out under the recorded inputs and compared with every later sample (position, wrapped heading and twist): the mean over the window of the squared error summed over the six channels, averaged over the windows that stay finite. The objective takes one reparameterized weight draw per step. β rises linearly to its maximum over an annealing period, and N_{obs} counts the training windows times their intervals. The gradient is a hand-written reverse pass through every step of the window: the 3×3 implicit solves, the SPD parameterization, the random-feature GPs, and ∇V's own derivative in position. The optimizer is LieSPHGP's:

- the whole gradient is clipped to global norm 1;
- Adam, with a cosine-decayed learning rate, updates every leaf except the levels: the GP weights, and the noise scales, although the rollout objective gives the noise scales no gradient (below);
- Prodigy, one per subnetwork, updates the levels.

The command-line tool `cvc ph-train` fits a model to plant-log runs and writes it as JSON (format `cvc-ph-se2/1`).

The rollouts in this objective run the drift alone, with no noise increment. So the objective fits M^{-1}, D, G and V, but Σ gets no gradient from it, and there is no observation model. LieSPHGP's own trainers fit those with an error-state extended Kalman filter: they minimize the filter's negative log marginal likelihood plus the KL term, a negative ELBO, which learns Σ and the observation noise. That stage is not yet ported. The model format already carries the observation-noise parameters it will need.

6.6 Steering: a port in the reference form, a modulation by option

The inconsistency. In the reference form every input is a port. In libcvc's plant logs the inputs are the vehicle's drive and its steering (`plant_log.h`), so the supply $y^\top u$ contains a steering term, $y_\delta u_\delta$. Section 4.3 shows that the vehicle's steering does no work on its body, and nothing in the learned structure forces that term to vanish. With its GP weights zero, the reference form reduces exactly to the linear-tire bicycle with steering as an input (`PhSe2.ReducesToTheLinearTireBicycle`), which is the truncation of Section 4.3.

The diagnostic. Split $y^\top u$ by input channel, using `output()`. Two measurements (transfix/libcvc #851):

- On the golden fit, steering is 13.4% of $|y^\top u|$ at its 16 states. With the drive input at zero, steering alone supplies energy at 10 of them (`PhSe2.TheReferenceFormBooksSteeringAsPower`).
- A fresh reference-form fit to the analytic plant's own flights still books 5.6% of $|y^\top u|$ through steering, and supplies energy by steering alone at 74 of 250 held-out steps, although the plant it learned from books none (`PhSe2Train.BothFormsFitTheDynamicBicycle`). That fit is good: its held-out rollout loss falls from 8.52 to 0.0006.

Steering as a modulation. libcvc #851 adds an opt-in form that mirrors Proposition 1: `se2_setup::modulation_dim` (0 by default). With `modulation_dim` $= 1$ the last input, the road-wheel angle δ, stops being a port. It is read by the GPs of D and G beside the twist:

$$D(v,\delta) = \mathrm{spd}\left(\ell_D + \mathrm{GP}_D(v/s_v,\delta)\right), \qquad G(v,\delta) = \ell_G + \mathrm{GP}_G(v/s_v,\delta), \qquad y = G(v,\delta)^\top v, \quad (25)$$

with u the drive alone in (19). The balance (21) holds unchanged, so with the drive at zero nothing is supplied, whatever δ is. Steering still steers, through D and G, as the steered tire's secant does in (9). The modulation is data, like the inputs, so the reverse pass drops its gradient.

- A modulated model is written as `cvc-ph-se2/2`; models without modulations stay `/1`, byte for byte.
- `cvc ph-train -steer-modulates` fits it from plant logs recorded with the actual road-wheel angle, which is `cvc-plant-log`'s default.
- The reference form stays the default and keeps its JAX parity. The modulated form has no JAX golden. It is held to central differences and to the analytic identities (Table 5).
- On the analytic plant's flights, it learns as well (held-out loss $7.63 \to 0.0092$) with steering's share at zero. The reference form fits those flights more closely, so the modulation is a matter of correct books, not of accuracy.
- At Adam's default learning rate of 3×10^{-3}, the reference form's gradient norm blew up (10^6–10^9) in all three runs tried on that data, while the modulated form converged in all four configurations tried. That is one dataset and one seed, so it is reported here, not claimed.

7 The navigator's energy identity and context work

The plant's books say where the *vehicle's* energy goes. The navigator has its own energy, $H_{\mathrm{nav}} = \frac{m}{2}\|v\|^2 + U_{\mathrm{nav}}$, on its point-mass surrogate (`integrate_surrogate_material`). libcvc #832 adds per-step books for that surrogate, and books across ticks for the deployed drive's potential.

7.1 The per-step identity

One semi-implicit step of a point mass m in a potential U held fixed over the step (coefficients frozen), with damping γ, a port force F and a speed clamp, is

$$v' = v + \tfrac{h}{m}(-g - \gamma v + F), \qquad \tilde{v} = \mathrm{clamp}_{v_{\max}}(v'), \qquad x' = x + h\tilde{v}, \qquad g = \nabla U(x). \quad (26)$$

With $H = \frac{m}{2}\|v\|^2 + U$, define the damping work $W_\gamma = \frac{h}{2}\gamma\, v\cdot(v+v')$, the port work $W_{\mathrm{port}} = \frac{h}{2}F\cdot(v+v')$, the clamp loss $W_{\mathrm{clamp}} = \frac{m}{2}(\|v'\|^2 - \|\tilde{v}\|^2) \geq 0$ and the clamp cross term $W_\times = h\, g\cdot(\tilde{v} - v')$. Then the residual is

$$r_E = \Delta H + W_\gamma + W_{\mathrm{clamp}} - W_{\mathrm{port}} - W_\times = \frac{h^2}{2}\Big[\tilde{v}^\top \bar{\mathcal{H}}\, \tilde{v} - \frac{1}{m}\, g\cdot(g + \gamma v - F)\Big], \quad (27)$$

exactly, with $\bar{\mathcal{H}} = 2\int_0^1 (1-s)\nabla^2 U(x + s\,\Delta x)\, ds$ the averaged Hessian. (Expand $\frac{m}{2}(\|v'\|^2 - \|v\|^2) = \frac{h}{2}(-g - \gamma v + F)\cdot(v + v')$, and Taylor-expand $U(x') - U(x) = h\, g\cdot\tilde{v} + \frac{h^2}{2}\tilde{v}^\top\bar{\mathcal{H}}\tilde{v}$.) `book_step` books any such step from its quantities. `rollout_energy` carries the books for every step of the material surrogate. They are evaluated in double, so they resolve terms of order h^3 below float round-off on U.

7.2 Two checks

- *The cheap bound*, every step:

$$|r_E| \le \kappa_H \frac{h^2}{2}\Big[\frac{|g|\,(|g| + \gamma|v| + |F|)}{m} + \hat{L}\,|\tilde{v}|^2\Big],$$

  with the secant Lipschitz estimate $\hat{L} = |\nabla U(x') - \nabla U(x)|/|x' - x|$ and safety factor $\kappa_H = 2$ (`energy_residual_bound`). It is a tolerance model, not a theorem, since $\hat{L}$ estimates the curvature and does not bound it. It catches work of order h left out of the books: an unlogged force, a stale field, an unbooked clamp.

- *The sharp check*, when asked (two extra force evaluations a step): $\hat{r}_E = \frac{h^2}{2}\big[\tilde{v}^\top \nabla^2 U(x)\tilde{v} - g \cdot (g + \gamma v - F)/m\big]$, with $r_E - \hat{r}_E = O(h^3)$. It catches a second-order error the identity does not predict. Explicit Euler ($x' = x + hv$) is the example: its r_E sits $h^2 g \cdot (g + \gamma v - F)/m$ above $\hat{r}_E$. That is second order, the same order as the bound, so explicit Euler passes the bound and fails the sharp check.

The explicit-Euler case matters in practice. GRL-SNAM's guide notes that `cvc::nav`'s native multi-start rollout runs explicit Euler by default, where the Python reference runs semi-implicit. On circular orbits of a goal spring, `NavPotential.ExplicitEulerRaisesTheAlarm` shows explicit Euler gaining energy at every step, by $\omega^2 h^2$, with the sharp check at half the bound on every step. The semi-implicit step meets the sharp check to 3.5×10^{-5} of the bound.

7.3 Context work and projection work

Across a tick of the deployed drive, U_{nav} is not fixed. The occupancy map is re-sensed, the field rebuilt, the coefficients re-predicted, and the carrot moved. Those changes do work on the navigator's energy that no step's books see. `sim_world` books them in its drive telemetry while it captures:

- `potential_end`: U at the pose the tick ends at, against the tick's field;
- `projection_work`: U's change across anything that moves the agent after the drive, the kinematic override and the `min_gap` de-overlap: `potential_end` minus U where the drive (or the plant) left the agent;
- `context_work`: U at the tick's start pose with this tick's context, minus U at the same pose with last tick's context (coefficients, carrot, field, material planes), evaluated as `step()` begins.

When nothing moves the agent between ticks, `context_work` equals this tick's starting U minus last tick's `potential_end`, bit for bit. The potential's change over a run then telescopes:

$$U_K^{\text{end}} - U_1^{\text{start}} = \sum_{k=1}^{K} (U_k^{\text{drive}} - U_k^{\text{start}}) + \sum_{k=1}^{K} \text{projection}_k + \sum_{k=2}^{K} \text{context}_k, \tag{28}$$

where U_k^{drive} is U where the drive left the agent on tick k. The first sum is what the drive's motion did under a fixed context. The other two are what the runtime did to the landscape. The drive itself leaves the three fields zero. The bicycle and dynamic rollouts are not point masses, so they reject requests for per-step point-mass books (they throw). For a vehicle, the plant's books of Section 5 are the counterpart.

7.4 Two sets of books, not one

When a navigation world drives the dynamic plant, there are two sets of books. The plant books the vehicle's kinetic energy (18). The telemetry books the navigator's potential (28). They are not one conserved quantity. U_{nav} is a learned landscape whose coefficients change every tick, not a physical energy of the vehicle. The two meet only through the controller, which is not a power-conserving interconnection. Keeping both, with their own residuals, is what makes each one checkable.

8 GRL-SNAM's side

8.1 The twins and their parity tiers

GRL-SNAM keeps Python twins of libcvc's vehicle and potential code. libcvc writes a golden file from its own test and GRL-SNAM copies it into `tests/fixtures`. The twin is then held to the golden at a stated tier:

- *double*: both sides in float64 from the same parameters, to 10^{-9} relative;
- *float*: libcvc in float32 against the twin in float32, to about 10^{-5};
- *mixed*: libcvc's float rollouts against the twin in float64, to 10^{-4} on values and 10^{-3} on gradients.

The torch twins differentiate with autograd, where libcvc uses forward-mode duals or hand-written reverses. So the gradient parity compares two independent derivations of one model. Table 1 lists the twins on `main`.

The JAX reference for the learned plant runs the other way. The $SE(2)$ vehicle package built on `CVC-Lab/LieSPHGP` (Section 6) generated libcvc's golden (`ph_se2_golden.json`) once, and the C++ port is held to it.

The torch tests need only torch and the copied goldens, not `pycvc`. (Two tests in `test_material_ext_parity.py` also hold `pycvc`'s bindings to the golden, and skip when the installed `pycvc` lacks the external-force input.)

8.2 What is not twinned yet

The pH form's parts, rates and step books are twinned in CVC-Lab/GRL-SNAM #138, held to libcvc's `DynBikePH.Golden`; the tangent `dyn_ph_tangent_D` is not. The navigator's point-mass books (`book_step`, the cheap bound, the sharp check and `rollout_energy`'s book fields) are twinned in #139, held to libcvc's `NavPotential.BooksGolden`. The tick telemetry (`potential_end`, `projection_work`, `context_work`) is a running world's, not a function's, so it has no twin. libcvc #857 binds it in `pycvc` instead: `nav_sim_world_set_capture_drive_telemetry` and `nav_sim_world_drive_telemetry` read a `sim_world`'s last tick, the three books included, from the first `pycvc` release that carries it. `pycvc` reports a world's plant kind (`nav_sim_world_plant_kind` returns `"dynamic"`), but not the plant's books. #831 left `dyn_plant`'s outputs untouched. libcvc #852's wheel-frame front slip changes them, and #138 moves the twin's plant with it, along with its three fixtures (`dyn_bike_golden.json`, `dyn_bike_ph_golden.json`, and the dynamic case of `material_ext_golden.json`).

8.3 Using the plant and its books from GRL-SNAM

What follows is what GRL-SNAM can do with what is merged today, and the small pieces it would add. Pieces not yet built are marked as proposals.

Twin	libcvc reference; golden; test	Tier and gate	Added
`grl_snam/dyn_bike.py` `dyn_plant`	`diff::dyn_plant` and its dual-number VJP; `dyn_bike_golden.json` (`DynBike.Golden`, 7 cases: cruise, hard corner, braking in a turn, past the peak, reversing, creeping, pulling away); `tests/test_dyn_bike_parity.py`	double: next state and VJP to 10^{-9} relative	GRL-SNAM #121; golden #134; wheel-frame slip #138
`grl_snam/dyn_bike.py` `dyn_ph_parts_at`, `dyn_ph_rates_at`, `dyn_plant_books`	`dyn_ph_parts_at`, `dyn_ph_rates_at`, `dyn_plant_books`; `dyn_bike_ph_golden.json` (`DynBikePH.Golden`, the 7 cases and a stop at the speed clamp); `tests/test_dyn_bike_ph_parity.py`	double: parts, rates, books and the books' gradient to 10^{-9} relative	#138; golden libcvc #848
`grl_snam/dyn4.py` `dyn4_plant`	`diff::dyn4_plant` (four wheels, suspension); `dyn4_golden.json` (11 cases); `tests/test_dyn4_parity.py`	double, 10^{-9}	#123
`grl_snam/ext_rollout.py` `dyn4_rollout_ext`	`diff::dyn4_rollout_ext`, with an external force; `dyn4_ext_rollout_golden.json`; `tests/test_dyn4_ext_rollout_parity.py`	mixed: states 10^{-4}, gradients 10^{-3}	#136
`grl_snam/material_bike.py` `integrate_dynamic_material`	`integrate_dynamic_material`; the dynamic case of `material_ext_golden.json`; `tests/test_material_ext_parity.py`	mixed: forward 10^{-4}, gradients 10^{-3}	#136; regenerated #138
`grl_snam/nav_potential.py`	`detail/nav_potential.h`; `nav_potential_golden.json` (`NavPotential.Golden`: barrier sweeps, U along a material rollout, `drive_potential`); `tests/test_nav_potential_parity.py`; also $F = -\nabla U$ by autograd	float: 2×10^{-5} (5×10^{-5} along the rollout)	#133
`grl_snam/nav_potential.py` `book_step`, `energy_residual_bound`, `material_energy`; `integrate_surrogate_material`'s books	`book_step`, `energy_residual_bound`, `rollout_energy`'s book fields; `nav_energy_books_golden.json` (`NavPotential.BooksGolden`: the bound on a sweep, a `diff::surr_step` rollout whose speed clamp cuts every step, the material surrogate with a port force and the sharp check, an unlogged port); `tests/test_nav_energy_books_parity.py`	double: `book_step` on libcvc's arguments to 10^{-12}; float: books to 10^{-5} of the step's bound (2×10^{-5} clamped)	#139; golden libcvc #849
JAX reference: the $SE(2)$ package built on `CVC-Lab/LieSPHGP` at `65db994` (not public)	that package against `cvc::ph::se2_vehicle` and `se2_train`; `ph_se2_golden.json`; libcvc `ph_se2_test`, `ph_se2_train_test`	double: one step 10^{-13}, rollout 10^{-12}, gradient 10^{-10}	libcvc #758

Table 1: GRL-SNAM's twins of libcvc's vehicle and potential code, and the JAX reference for the learned plant. GRL-SNAM numbers are CVC-Lab/GRL-SNAM pull requests.

Train through the dynamic plant. The dynamic rollouts are twinned and differentiable: `integrate_dynamic_material` for the material bicycle, and `dyn4_rollout_ext` for the four-wheel vehicle with an external force. A trainer that today rolls out the kinematic bicycle can roll out `dyn_bike.dyn_plant` in its place, under the same `bike_control` commands, and backpropagate through tires, load transfer and actuator lag. In C++ the same choice is `coef_train`'s `rollout_kind::dynamic`. In a live world it is `plant_host.use(sim, "dynamic")`.

Book the energy. The torch twin of the books (#138) is small, because `dyn_bike.py`'s step already computes every part: the axle forces $F_{x,w}$, the lateral forces $F_{y,w}$, the secants S_w and the forward speeds u_w. The rates at a state (`dyn_ph_rates_at`) are

```
nf = (-sin d, cos d, lf cos d);  nr = (0, 1, -lr)
tf = (cos d, sin d, lf sin d);   tr = (1, 0, 0)
R  = cx e_x e_x^T + Sf nf nf^T + Sr nr nr^T
supplied   = v . (tf Fxf + tr Fxr)          # y^T u
dissipated = v^T R v                        # >= 0
floor      = v . (nf (Fyf + Sf nf.v) + nr (Fyr + Sr nr.v))   # round-off
grade      = -m g theta vx
```

The books of a step (`dyn_plant_books`) are the trapezoid of these between the start (with the step's actuator outputs) and the pre-limiter twist, plus H's change across the limiters (Section 5). `tests/test_dyn_bike_ph_parity.py` holds them to libcvc's `DynBikePH.Golden` in double: the worst relative difference is 7.1×10^{-14} on the books and 4.4×10^{-14} on their gradient, which autograd takes and libcvc takes by dual numbers.

Book the navigator's energy. The point-mass surrogate's books come with its rollout. Given `energy={"books": True, "sharp": True}`, `integrate_surrogate_material` fills `kinetic`, `w_damp`, `w_port`, `w_clamp`, `w_cross`, `residual`, `bound` and `residual_sharp` for every step, in float64 and detached, as libcvc's `rollout_energy` does. The rollout's outputs are unchanged bit for bit. `nav_potential.book_step` books any other such step; the parity test books `sdf_rollout`'s clamped surrogate with it. `tests/test_nav_energy_books_parity.py` holds the books to `NavPotential.BooksGolden`. `book_step` on libcvc's arguments agrees bit for bit, and the books agree to 1.1×10^{-6} of the step's bound on the clamped rollout and 8.3×10^{-8} on the material surrogate. The books see only the integrator when the patch's gradient channels are its values' gradients. Otherwise each step also books the gap between the force and $-\nabla U$, work of order h, which the cheap bound reads as a stale field: on `NavPotential.Golden`'s curved patch it fills 99.65% of the bound at $h = 0.002$. Across ticks, a live world's books come from `pycvc` (libcvc #857). With capture on, `nav_sim_world_drive_telemetry` gives each agent's `projection_work` and `context_work`, the terms of the last two sums of (28).

Energy-aware objectives (proposal). With the books available inside the rollout, a trainer can price energy directly:

- $\sum \max(y^\top u, 0)\, h$, the drive energy, as a cost;
- the dissipated energy $\sum v^\top \mathsf{R} v\, h$ as a proxy for tire scrub and sliding (it grows sharply past the peak);
- the limiter energy as a penalty for leaning on the speed clamp.

Each is differentiable almost everywhere, as the plant is. Because steering does no work, the drive energy counts every joule the policy puts into the vehicle.

Health checks. The books double as run-time checks of a rollout:

- the residual's share of the flow (in the tests' runs, 1.1×10^{-4} to 1.2×10^{-3} at $h = 0.01$ s) flags a step too long for the motion;
- a floor line above round-off flags a plant whose slips no longer root at $n_w^\top v = 0$;
- a limiter line that dominates flags a policy that lives at the speed clamp.

On the navigator side, the cheap bound flags an unlogged force or an unbooked clamp, and the sharp check flags an explicit-Euler integrator.

New force terms are ports. GRL-SNAM's research extensions add force terms to the navigator: material and terrain costs, and external forces. Section 4.8 gives the rule. A conservative term goes in U_{nav} with its force $-\nabla U$, which the twin's autograd gradcheck verifies. A non-conservative term is a port, booked as W_{port} on the surrogate. On the vehicle, it is either already in $y^\top u$ (if it acts through the controller) or a new column G_e (if it acts on the body).

9 Validation

Tables 2–6 map each claim to the test that checks it. They give the tolerance the test enforces and the measured value. The commit is the squash commit of the pull request that added or last tightened the test.

Where the values were measured (Windows, MSVC 19.44, double unless stated; Appendix A gives the exact setup):

- libcvc `c1f104c0`: all 62 tests of the five executables passed. The tables list the ones this paper relies on.
- The rows for #851: its branch commit, where the same executables' 69 tests passed (plus the CLI's four `PhTrain` tests).
- GRL-SNAM `0e21085` (torch 2.8 CPU): 53 tests passed and 2 were skipped for the `pycvc` reason above; none failed.
- The rows for libcvc #852 and GRL-SNAM #138: libcvc's branch commit `1a0acc77`, where `nav_dyn_bike_test`'s 23 tests and `physics_dyn_bike_plant_test`'s 3 passed, and so did the ten other suites that use the plant; and GRL-SNAM's `2f76ab0`, #138's squash commit, where the dynamic twins' parity tests passed (57 passed, 6 skipped for `pycvc`).

Appendix A gives the exact versions, the commands that produced these values, and the lines each test prints.

10 Limitations and future work

The four-wheel vehicle in pH form. libcvc's two-track vehicle (`inc/cvc/nav/detail/dyn_4w.h`, twinned as `grl_snam.dyn4`) has four contact patches with Ackermann front wheels, spring-damper corners, and heave, roll and pitch. Its pH form is not yet written. Since libcvc #852 the single track measures its slips as that model does, each wheel's sideways velocity in its own frame, $w_i = n_i^\top v$, with each slip's root at $n_i^\top v = 0$. So the single track's form should carry over directly: $f_\varepsilon = 0$ but for

Test	What it shows	Tolerance	Measured	Commit
`DynBikePH.TheStructure IsPortHamiltonianAt EverySlip`	J skew, R symmetric with non-negative coefficients, $w^\top \mathsf{R} w \geq 0$, $H = \frac{1}{2} p \cdot v$; no supply without axle force (Prop. 1, Sec. 4.3); 500 states at each of $\mu = 0.9, 0.6$, steer to 0.5 rad, forward and reversing	exact for skew, symmetry and zero supply; $v^\top \mathsf{J} v$ to 10^{-9}; at least 100 states past a peak	holds at all 1000 states; 244 of 500 past a peak at each μ	`950154b` (#831)
`DynBikePH.TheVector FieldIsThePlantsAtEvery Slip`	(12) is the plant's vector field: a 10^{-5} s `dyn_plant` step against `dyn_ph_rhs`, flat and on 3%, forward and reversing, 400 states per μ	10^{-3} of mg (mgL on yaw)	worst $1.6 \times 10^{-4} mg$ ($\mu = 0.9$), $1.1 \times 10^{-4} mg$ ($\mu = 0.6$)	`950154b` (#831)
`DynBikePH.StepBooks CloseToSecondOrder`	the books (17) close to second order per step (Prop. 2); the limiters are charged when braking to a stop in a turn	per-halving ratio in $(3, 5.5)$; summed residual below 5×10^{-3} of the flow for $h \leq 0.01$	ratios 3.83, 3.92, 3.95; 1.2×10^{-3} at $h = 0.01$; braking: limiter -358 J, residual 4×10^{-6}	`950154b` (#831), `1a0acc77` (#852)
`DynBikePH.TheFloorTerm IsRoundOff`	f_ε is the secants' rounding (Sec. 4.4)	10^{-12} of the flow, $\varepsilon = 0.5$ to 10^{-9}	at most 2.2×10^{-16}	`1a0acc77` (#852)
`DynBikePH.TheTangentIs TheTiresDerivative`	(14)–(15) is $-\partial(Q_y, Q_r)/\partial(v_y, r)$ at 300 states	10^{-6} (central differences)	1.4×10^{-7}	`1a0acc77` (#852)
`DynBikePH.TheSecant StaysPSDWhereTheTangent DoesNot`	tangent indefinite past the peak, secant never (Sec. 4.6)	tangent PSD below $0.5\mu g$; indefinite somewhere above; rear never past peak; secant never negative	as stated at $\mu = 0.9$ and 0.6; indefinite in 25–50% of samples above $0.5\mu g$	`950154b` (#831)
`DynBike.Golden`	`dyn_plant`'s outputs: unchanged by the books (#831), regenerated for the wheel-frame slip (#852)	golden values	holds	`1e3ed0b` (#714), `1a0acc77` (#852)
`DynBike.KinematicLimit FollowsBikeStep`	the kinematic bicycle is the stiff-tire limit, with the mass at the rear axle	worst gap below 0.2 of the vehicle radius; closed-loop median below 0.1 of it	within 0.011 of the kinematic path (mean path 5.6)	`1e3ed0b` (#714)

Table 2: The analytic plant: its pH form and its books (libcvc's `nav_dyn_bike_test`). The flow is the sum of the magnitudes of every line of the books. #714 is the plant itself, which predates the pH work. Before #852 (on `c1f104c0`) braking measured -354 J and 7×10^{-7}, and the floor test, then `TheFloorTermVanishesWithTheFloor`, measured $5.4 \times 10^{-4} \to 5.4 \times 10^{-6} \to 5.4 \times 10^{-8}$; the other rows' values are unchanged.

Test	What it shows	Tolerance	Measured	Commit
`DynBikePlant.StepsAsTheDynamicPlantDoes`	the live plant's ticks are `dyn_plant`'s substeps	bit for bit	bit for bit	`eb9187c` (#833)
`DynBikePlant.TheBooksClose`	(18) closes on two vehicles over 30 s of weaving and braking	identity 10^{-9} of the flow; residual 5×10^{-3}	residual 1.1×10^{-4} (full-size), 8.9×10^{-4} (1/10 scale) of the flow	`eb9187c` (#833)
`DynBikePlant.PlantHostPutsTheAgentsOnItByName`	`plant_host` puts a world on `"dynamic"` with no physics world; its books close	every agent reaches its goal; books as above	3 agents reach goals 120 m on in 11.5 s	`eb9187c` (#833)

Table 3: The live plant (libcvc's `physics_dyn_bike_plant_test`). Its floor line is 5×10^{-15} J over the full-size run since #852 (1.1 J before); the residuals are unchanged.

Test	What it shows	Tolerance	Measured	Commit
`PhSe2.MatchesJaxInDouble`, `PhSe2.MatchesJaxInFloat`	the C++ model is the JAX reference's: every piece and one step at 16 states (a 64-feature, 1,691-parameter fit)	10^{-13} relative (double); 2×10^{-6} (float)	1.9×10^{-16}; 1.1×10^{-7}	#758
`PhSe2.RolloutMatchesJax`	a 25-step rollout matches	10^{-12}	3.1×10^{-15}	#758
`PhSe2.EnergyBalanceIsExact`	(21) at every golden state; $y^\top u$ from `output()` is the supply	10^{-11}; 10^{-12}	holds	#758, `950154b` (#831)
`PhSe2.NoisePowerIsTheEnergyTheDiffusionAdds`	(22): antithetic Monte Carlo of H's mean gain over one 10^{-4} s step	4 standard errors +2%	3.585×10^{-6} J against 3.564×10^{-6} J predicted	`950154b` (#831)
`PhSe2.StepIsSecondOrder`	the Lie-IMEX step (23) is second order	ratio in $(3.5, 4.5)$ per halving	3.99, 3.99, 4.00	#758
`PhSe2.ReducesToTheLinearTireBicycle`	with the GPs off, the drift is the linear-tire bicycle with steering as an input	10^{-9}	holds	#758
`PhRff.FeaturesApproachTheMaternKernel`	(20) approximates the Matérn kernel	0.03	holds	#758
`PhSe2Train.GradientMatchesJax`	the hand-written reverse pass is JAX's autodiff of (24)	10^{-10} of each leaf's largest entry	8.1×10^{-15} over 1,691 entries	#758
`PhSe2Train.GradientMatchesFiniteDifferences`	the same against central differences on a small model	2×10^{-5}	5.6×10^{-6}	#758
`PhSe2Train.OptimizerMatchesOptax`	clip, Adam and Prodigy match the reference optimizer over 300 steps	10^{-12}	1.6×10^{-16}	#758
`PhSe2Train.ShortFitLowersTheLoss`	a 1,200-step fit to trajectories simulated from a second random model generalizes to held-out ones	held-out loss below 0.1×	$1.732 \to 0.077$ (22×)	#758

Table 4: The learned plant (libcvc `ph_se2_test`, `ph_se2_train_test`). libcvc #758 merged as `17adc93`, with the model in `0fa251b` and the trainer in `63651e8`.

Test	What it shows	Tolerance	Measured	Commit
`PhSe2.TheReferenceForm BooksSteeringAsPower`	in the reference form, steering alone supplies power (Sec. 6.6)	nonzero	share 0.134; supplies energy at 10 of 16 states	#851
`PhSe2.ModulatedSteering SuppliesNoPower`	(25): the balance holds; no supply at zero drive, whatever δ; steering still changes the motion	balance 10^{-11}; supply exactly 0	holds at 200 states; a 0.5 steer step changes the twist's rate by at least 0.18	#851
`PhSe2.ModulatedModel RoundTripsExactly`, `PhSe2. ModulatedStepIsSecond Order`	the `/2` file round-trips; models without modulations stay `/1`; the step stays second order	exact; ratio in $(3.5, 4.5)$	exact; 4.29, 4.08, 4.02	#851
`PhSe2Train.Modulated GradientMatchesFinite Differences`, `PhSe2Train. ModulatedShortFitLowers TheLoss`	the modulated reverse pass is the gradient; the modulated form trains	2×10^{-5}; held-out loss below $0.1\times$	3.9×10^{-7} over 68 entries; $1.417 \to 0.070$	#851
`PhSe2Train.BothFormsFit TheDynamicBicycle`	both forms fit the analytic plant's flights; only the reference form books steering power	held-out loss below $0.01\times$; share > 0 (reference), $= 0$ (modulated)	reference $8.52 \to 0.0006$, share 0.056, supplies energy at 74 of 250 steps; modulated $7.63 \to 0.0092$, share 0	#851

Table 5: The learned plant's steering (Section 6.6): transfix/libcvc #851, measured on its branch (Table 7).

the secants' rounding, and $\mathsf{R} = c_x e_x e_x^\top + \sum_i S_i n_i n_i^\top$. The suspension springs would add their stored energy (and a height-dependent gravity term) to H, and the dampers would add to R. Writing it out, and gating it as Table 2 gates the single track, is the next piece of this line of work.

Structure-preserving integrators. Neither plant's integrator keeps a discrete energy identity. Both are first order (analytic) or second order (learned) with a booked residual. A discrete-gradient or implicit-midpoint step would close the books exactly. It would cost a nonlinear solve per step, and the analytic plant's stiff-tire stability at the navigator's tick would have to be re-established.

The learned plant's evidence bound. The rollout objective does not fit the diffusion or an observation model. LieSPHGP's EKF-based ELBO is the next stage for `cvc::ph`. Beyond the synthetic comparison of Section 6.6, the fit's accuracy against the analytic plant is not reported here.

The learned plant's steering, upstream. The modulated form of Section 6.6 exists only in libcvc. It has no JAX golden, and whether LieSPHGP or the $SE(2)$ package adopts it is for their authors to decide.

The closed loop. The navigator commands the vehicle through a path-following controller, not an energy-shaping one, so the closed loop is not port-Hamiltonian, and the two sets of books of

Test	What it shows	Tolerance	Measured	Commit
`NavPotential.Surrogate EnergyIdentityPerStep`	(27): r_E is second order and $r_E - \hat{r}_E$ third, as h halves $0.02 \to 0.0025$, with damping and a port force	orders 2 ± 0.2 and 3 ± 0.3; $\lvert r_E \rvert \leq$ bound at every step	1.99, 2.00, 2.00; 2.99, 2.94, 2.93	`09fa876` (#832)
`NavPotential.ClampWork ClosesTheBooks`	with a hard speed clamp, the books close only with $W_\times$ (and W_{clamp})	every clamped step within the bound; 90% fail without $W_\times$	213 of 213 close; all 213 fail without it	`09fa876` (#832)
`NavPotential.Unlogged ForceRaisesTheAlarm`	an unbooked port force fails the cheap bound	at least 95% of steps	229 of 240	`09fa876` (#832)
`NavPotential.Explicit EulerRaisesTheAlarm`	explicit Euler gains energy every step and fails the sharp check; semi-implicit meets it	sharp check above 0.3 of the bound (explicit), below 10^{-3} (semi-implicit)	0.50; 3.5×10^{-5}	`09fa876` (#832)
`NavPotential.SimWorld BooksTheContextWorkOf AGoalChange`	a goal change between ticks is context work; the books close across the tick; capturing changes no output	exact equality; byte-identical positions	holds (retargeted agents: -1.63, -3.27)	`09fa876` (#832)
`NavPotential.SimWorld BooksTheMinGapPushAs ProjectionWork`	`min_gap`'s push is projection work, zero for agents it does not move	nonzero for the pair, exactly zero otherwise	$+0.18$, $+0.59$; others 0	`09fa876` (#832)
`NavPotential.*ForcesAre MinusGradU*`, `Nav Potential.DriveForce*`	each call site's conservative force is $-\nabla U_{\text{nav}}$ (central differences)	site-specific	worst 5.7×10^{-4} relative on exact planes	`7d47b98` (#747)
`test_dyn_bike_parity.py`	the torch plant and its autograd gradient are libcvc's (7 cases, a batch, braking distance)	10^{-9} relative	9 passed (also on #138's wheel-frame slip)	GRL-SNAM `d75eecc` (#121)
`test_dyn_bike_ph_parity.py`	the torch pH parts, rates and step books, and the books' autograd gradient, are libcvc's (8 cases, a batch, and second-order books on a swerve)	10^{-9} relative	18 passed; worst 7.1×10^{-14}	`2f76ab0` (#138)
`test_nav_potential_parity.py`	the torch U_{nav} is libcvc's; its forces are $-\nabla U$ by autograd	2×10^{-5}; 5×10^{-5} along the rollout	8 passed	`7e4195d` (#133)
`test_dyn4_parity.py`	the four-wheel torch plant is libcvc's	10^{-9}	14 passed (11 cases)	`5b8e21f` (#123)
`test_dyn4_ext_rollout_parity.py`, `test_material_ext_parity.py`	the dynamic rollouts with an external force, forward and reverse	10^{-4} forward, 10^{-3} gradients	22 passed, 2 skipped	`0e21085` (#136)

Table 6: The navigator's books (libcvc `nav_potential_test`) and GRL-SNAM's twins (CVC-Lab/GRL-SNAM `tests/`). GRL-SNAM numbers are its pull requests.

Section 7.4 stay separate. Interconnection and damping assignment for a car is a research problem: with two inputs and three velocity degrees of freedom, its matching condition fails in general [9].

Python access. The plant step's books and the navigator's point-mass books have torch twins (GRL-SNAM #138 and #139), and `pycvc` reads a running world's tick telemetry (libcvc #857). The live plant's books are still C++ only: `pycvc` bindings for `dyn_bike_plant`'s books would let GRL-SNAM's evaluations read them from a running world. (`pycvc` binds neither `dyn_bike_plant` nor `plant_host` today.)

A Reproducing the validation tables

A.1 What was run

Piece	Version
libcvc	`master` at `c1f104c0`; its `nav`, `ph` and `physics` sources and tests are identical to the v3.7.1 tag (`d8ca602b`). The #851 rows: its branch commit `6e7ab557`, cut from `master` at `cc8800a8`. The #852 rows (and every `nav_dyn_bike_test` and `physics_dyn_bike_plant_test` line below): its branch commit `1a0acc77`, on #848's head `d1a2a78e`, cut from `master` at `72e4c607`.
GRL-SNAM	`main` at `0e21085` (`3a38ae4` adds only this paper). The #138 rows: `main` at `2f76ab0`, #138's squash commit
Operating system	Windows 11 Pro 10.0.26100, x86-64
Compiler	MSVC 19.44 (Visual Studio 2022, toolset 14.44.35207), Release
Build tools	CMake 3.31.7, Ninja 1.12.1, cvcpkg 2.5.6
Dependencies	`cvcpkg install-deps` of libcvc's recipe: 38 packages for windows/x86_64/release/shared, among them Boost, googletest and HDF5
Python	3.11.13, torch 2.8.0+cpu, numpy 2.4.6, pytest 9.1.1

Table 7: The setup behind every measured value in Tables 2–6.

A.2 libcvc

From a libcvc checkout at v3.7.1 or later, in a Visual Studio 2022 x64 developer prompt (or after `vcvars64.bat`), the commands used were these, with the paths shortened:

```
cvcpkg install-deps cvcpkg\recipes\libcvc --prefix deps --config release
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=%CD%\deps ^
      -DCVC_BUILD_TESTS=ON -DCVC_BUILD_CVCGL=OFF -DCVC_ENABLE_CUDA=OFF ^
      -DCVC_BUILD_PYCVC=OFF -DCVC_BUILD_PYCVC_GL=OFF -DCVC_WITH_JOLT=ON
cmake --build build --target nav_dyn_bike_test physics_dyn_bike_plant_test ^
      ph_se2_test ph_se2_train_test nav_potential_test
set PATH=%CD%\deps\bin;%CD%\build\bin;%PATH%
for %t in (nav_dyn_bike_test physics_dyn_bike_plant_test ph_se2_test
           ph_se2_train_test nav_potential_test) do build\bin\%t.exe
```

The extra CMake flags only switch off what these tests do not need: the graphics library, CUDA and the Python bindings. `CVC_WITH_JOLT=ON` builds the rigid-body backend that `physics_dyn_bike_plant_test` links against, although the dynamic plant itself needs no physics world. If CMake finds no Python 3, add `-DPython3_EXECUTABLE=`*`path`*.

On Linux or macOS the same steps are

```
cvcpkg install-deps cvcpkg/recipes/libcvc --prefix deps --config release
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=$PWD/deps \
      -DCVC_BUILD_TESTS=ON -DCVC_BUILD_CVCGL=OFF -DCVC_ENABLE_CUDA=OFF \
      -DCVC_BUILD_PYCVC=OFF -DCVC_BUILD_PYCVC_GL=OFF -DCVC_WITH_JOLT=ON
cmake --build build --target nav_dyn_bike_test physics_dyn_bike_plant_test \
      ph_se2_test ph_se2_train_test nav_potential_test
for t in nav_dyn_bike_test physics_dyn_bike_plant_test ph_se2_test \
         ph_se2_train_test nav_potential_test; do ./build/bin/$t; done
```

These were not run verbatim for this paper, since no Linux machine was available where it was written. libcvc's CI builds every test and runs it under CTest on Linux, macOS and Windows for each pull request, #851 included. The tests are registered with CTest under their Google Test names, so `ctest -test-dir build -R 'DynBike|PhSe2|PhRff|PhMat3|PhRng|NavPotential'` runs these suites. That is a superset of the tables' rows.

A.3 What the libcvc tests print

Each test prints its measured values. These are the lines behind the tables, from the run of Table 7, with long lines cut at "...". Lines that name the vehicle preset are summarized rather than quoted.

Other compilers and platforms can differ in the last digits. The training runs (the held-out losses) can differ more, since a long optimization amplifies rounding. The tolerances in the tables are what the tests enforce.

```
nav_dyn_bike_test (23 tests, at #852's 1a0acc77):
  kinematic limit, 200 ticks (mean path 5.624), bike_step's commands through the plant:
    the kinematic bicycle's vehicle: within 0.01074 of its path (mean worst 0.00185)
  mu 0.9: 244 of 500 states have a tire past its peak
  mu 0.6: 244 of 500 states have a tire past its peak
  vector field vs dyn_plant at every slip, mu 0.9: worst |dp/dt - pH| = 1.62e-04 m g
  vector field vs dyn_plant at every slip, mu 0.6: worst |dp/dt - pH| = 1.06e-04 m g
  swerve past the peak, hdt 0.0200 s: ... mean |residual| per step 3.320e+00 J, |sum| / flow 2.24e-03
  swerve past the peak, hdt 0.0100 s: ... mean |residual| per step 8.664e-01 J, |sum| / flow 1.21e-03
  swerve past the peak, hdt 0.0050 s: ... mean |residual| per step 2.209e-01 J, |sum| / flow 6.25e-04
  swerve past the peak, hdt 0.0025 s: ... mean |residual| per step 5.593e-02 J, |sum| / flow 3.19e-04
  braking to a stop, hdt 0.0100 s: dH -1.084e+05 J, limiters -358.2 J, |sum residual| / flow 4.43e-06
  speed floor 5e-01 m/s: worst |floor power| / (dissipated + |supplied|) = 1.67e-16
  speed floor 5e-02 m/s: worst |floor power| / (dissipated + |supplied|) = 2.16e-16
  speed floor 5e-03 m/s: worst |floor power| / (dissipated + |supplied|) = 2.16e-16
  speed floor 1e-09 m/s: worst |floor power| / (dissipated + |supplied|) = 1.58e-16
  tangent vs central differences: worst 1.39e-07 of its scale
  (then the tangent's table by lateral g, at mu 0.9 and 0.6: 0% indefinite below 0.5 mu g)
  (on c1f104c0, before #852, 21 tests: the same lines, but braking -353.5 J and 7.23e-07,
   and speed floor 5.35e-04, 5.42e-06, 5.42e-08, 1.95e-16)
physics_dyn_bike_plant_test (3 tests, at 1a0acc77):
  RoboRacer, 30 s: drive 845.1 J, ..., floor 6.02e-16, ..., residual -1.49 (8.88e-04 of the flow)
  (and the full-size preset's line: floor -5.44e-15, residual 1.10e-04 of the flow; all agents
   reached in 11.5 s. On c1f104c0: RoboRacer 8.86e-04, full-size floor 1.14 J)
ph_se2_test (14 tests; 18 with #851):
  one step, double vs JAX float64: worst |d| / (1 + |x|) 1.87e-16 over 16 states
  one step, float vs JAX float64: worst |d| / (1 + |x|) 1.08e-07
  25-step rollout, double vs JAX: worst 3.13e-15
  noise energy over 1e-04 s: 3.5851e-06 J measured (se 2.6e-08), noise_power h = 3.5643e-06 J
  substeps x2: error 2.570e-06 -> 6.442e-07 (x3.99)       (and x3.99, x4.00)
  [#851] steering's share of |y.u| at the golden states 0.134; it supplies energy at 10 of 16 ...
  [#851] modulated, substeps x2: error 3.028e-04 -> 7.055e-05 (x4.29)   (and x4.08, x4.02)
ph_se2_train_test (8 tests; 11 with #851):
  reference: 71 entries against central differences: worst relative 5.57e-06
  1,691 entries against JAX: worst |d| / max|leaf| 8.05e-15
  300 optimizer steps against optax: worst 1.58e-16 at 13 checkpoints
  held-out rollout loss 1.7320 -> 0.0773
  [#851] modulated: 68 entries against central differences: worst relative 3.87e-07
  [#851] reference form on the dynamic bicycle: held-out rollout loss 8.5199 -> 0.0006;
         steering's share of |y.u| 0.056, supplying energy at 74 of 250 steps
  [#851] modulated form on the dynamic bicycle: held-out rollout loss 7.6348 -> 0.0092; ... 0.000
nav_potential_test (16 tests):
  books, dt 0.0200 -> 0.0100: order of r_E 1.987, of r_E - r_hat_E 2.993
  books, dt 0.0100 -> 0.0050: order of r_E 1.997, of r_E - r_hat_E 2.943
  books, dt 0.0050 -> 0.0025: order of r_E 2.000, of r_E - r_hat_E 2.930
  clamp: 213 steps (76 in the barrier); |r_E|/bound worst 0.473; without W_cross 213 fail; ...
  unlogged port: 229/240 steps raise the alarm; smallest |r_E + W_port|/bound 0.44
  explicit Euler, 157 steps: ... (r_E - r_hat_E)/bound from 4.99e-01, |.| up to 4.99e-01; ...
  semi-implicit Euler, 157 steps: ... (r_E - r_hat_E)/bound from -3.54e-05, |.| up to 3.54e-05; ...
  agent 0 (retargeted): context work -1.6294 (without the goal change +0.0000), U 73.1821 -> 71.5528
  agent 2 (retargeted): context work -3.2729 (without the goal change +0.0871), U 74.8257 -> 71.5528
  min_gap push: projection work +0.1842, +0.5935
```

A.4 GRL-SNAM

From a GRL-SNAM checkout of `main`, with torch installed and no other setup (the parity tests need the copied goldens, not `pycvc`):

```
python -m pytest -v -p no:cacheprovider tests/test_dyn_bike_parity.py \
    tests/test_nav_potential_parity.py tests/test_dyn4_parity.py \
    tests/test_dyn4_ext_rollout_parity.py tests/test_material_ext_parity.py
```

On Windows, run it from the checkout's root with `PYTHONPATH` set to it (`set PYTHONPATH=.`). The expected summary is `53 passed, 2 skipped`. The two skips are `test_material_ext_parity.py`'s `pycvc` checks, which skip when the installed `pycvc` lacks the material external-force input. As

GRL-SNAM's guide warns, a skipped test is not a verified one: read the `-v` listing. From `2f76ab0` (#138) on, add `tests/test_dyn_bike_ph_parity.py`, the pH twin's 18 tests, which all pass.

B Equation-to-code map

Concept	libcvc	GRL-SNAM
The plant's step, Sec. 3, 5	`diff::dyn_plant`, `dyn_plant_vjp` (`nav/detail/dyn_bike.h`)	`grl_snam.dyn_bike.dyn_plant`
Secant stiffness, Lemma 1	`dyn_secant_w`	`dyn_bike._secant_w`
H, p, J, (2)	`dyn_ph_hamiltonian`, `dyn_ph_momentum`, `dyn_ph_J`	`dyn_bike.dyn_ph_hamiltonian` (#138)
R, G, u, f_ε, (9)–(11)	`dyn_ph_R`, `dyn_ph_G`, `dyn_ph_input`, `dyn_ph_floor`; all parts in `dyn_ph_parts_at`	`dyn_bike.dyn_ph_parts_at` (#138)
The vector field (12); the balance (13)	`dyn_ph_rhs`; `dyn_ph_rates_at` (struct `dyn_ph_rates`)	the balance: `dyn_bike.dyn_ph_rates_at` (#138)
The tangent (14), (15)	`dyn_ph_tangent_D`	—
The step's books (17)	`dyn_plant_books` (struct `dyn_ph_books`); `dyn_step_free`	`dyn_bike.dyn_plant_books`; `dyn_plant(..., with_free=True)` (#138)
The live plant (18)	`cvc::physics::dyn_bike_plant`; `plant_host "dynamic"`	plant kind via `pycvc`
External force through the controller	`diff::ext_in`, `bike_control`; `nav/ext_rollout.h`	`grl_snam.ext_rollout`, `material_ext`
The learned plant (19)	`cvc::ph::se2_vehicle`, `se2_model`; its golden from the $SE(2)$ package built on `CVC-Lab/LieSPHGP`	—
Steering as a modulation (25)	`se2_setup::modulation_dim`, `cvc-ph-se2/2`, `cvc ph-train -steer-modulates` (#851)	—
Output, noise power, (21)–(22)	`se2_vehicle::output`, `mass`, `noise_power`	—
RFF features (20); SPD levels	`cvc::ph::rff`; `spd_from_levels` (`mat3.h`)	—
Lie-IMEX step (23)	`se2_vehicle::step`, `interval_step`, `rollout`	—
Rollout objective (24)	`se2_rollout_objective`, `se2_optimizer`; `cvc ph-train`	—
U_{nav}, (3)	`nav/detail/nav_potential.h`, `detail::material_energy`	`grl_snam.nav_potential`
Per-step identity (27); checks	`book_step`, `energy_residual_bound`; `rollout_energy`'s book fields	`nav_potential.book_step`, `energy_residual_bound`; `integrate_surrogate_material(energy=...)` (#139)
Context and projection work (28)	`drive_telemetry::potential_end`, `projection_work`, `context_work` (`sim_world`)	read via `pycvc nav_sim_world_drive_telemetry` (libcvc #857)

Table 8: Where each equation lives. "—" marks C++-only pieces.

References

[1] A. S. Ellendula, Y. Wang, M. Nguyen, and C. Bajaj. GRL-SNAM: Geometric reinforcement learning with path differential Hamiltonians for simultaneous navigation and mapping in

unknown environments. arXiv:2601.00116, December 2025.

[2] L. McLennan, T. Rapolu, and C. Bajaj. Stochastic port-Hamiltonian Gaussian processes for learning from noisy dynamics on Lie groups. Manuscript, CVC, The University of Texas at Austin, October 2026; code `CVC-Lab/LieSPHGP`.

[3] H. B. Pacejka. *Tire and Vehicle Dynamics*, 3rd edition. Butterworth-Heinemann, 2012.

[4] R. Rajamani. *Vehicle Dynamics and Control*, 2nd edition. Springer, 2012.

[5] A. Rahimi and B. Recht. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems 20*, 2007.

[6] J. E. Marsden and T. S. Ratiu. *Introduction to Mechanics and Symmetry*, 2nd edition. Springer, 1999.

[7] A. van der Schaft and D. Jeltsema. Port-Hamiltonian systems theory: An introductory overview. *Foundations and Trends in Systems and Control*, 1(2–3):173–378, 2014.

[8] A. J. van der Schaft and B. M. Maschke. On the Hamiltonian formulation of nonholonomic mechanical systems. *Reports on Mathematical Physics*, 34(2):225–233, 1994.

[9] R. Ortega, A. van der Schaft, B. Maschke, and G. Escobar. Interconnection and damping assignment passivity-based control of port-controlled Hamiltonian systems. *Automatica*, 38(4):585–596, 2002.